04 — Variablen

Werte benennen, speichern und wiederverwenden

HTW Berlin – Wirtschaftsingenieurwesen

WiSe 2026/27

Agenda

1. Wozu Variablen?
2. Zuweisung mit =
3. Unter der Haube: Name $\rightarrow$ Wert im Speicher
4. Neu zuweisen – und jeden Schritt tracen
5. Variablennamen: Regeln & Konventionen
6. f-Strings – Variablen im Text
7. **Klassiker: Variablen-Tausch & Mini-Bon** – live programmiert

Lernziel: Werte benennen und wiederverwenden – und **verstehen**, was bei `x = x + 1` im Speicher wirklich passiert.

Heute: nach jedem Konzept *Live-Demo* (ich) und *Sofort ausprobieren* (Sie, Notebook 04).

Wozu Variablen?

Ohne Variable

```
print("Eco-Sneaker kostet 89.95 EUR")
print("Eco-Sneaker hat 4.5 Sterne")
print("Eco-Sneaker liegt 42 mal auf Lager")
```

Mit Variable

```
produkt = "Eco-Sneaker"
print(f"{produkt} kostet 89.95 EUR")
print(f"{produkt} hat 4.5 Sterne")
print(f"{produkt} liegt 42 mal auf Lager")
```

Eine **Variable** gibt einem Wert einen **Namen**. Der Wert steht nur noch an **einer** Stelle – ändert er sich, stimmt automatisch alles wieder.

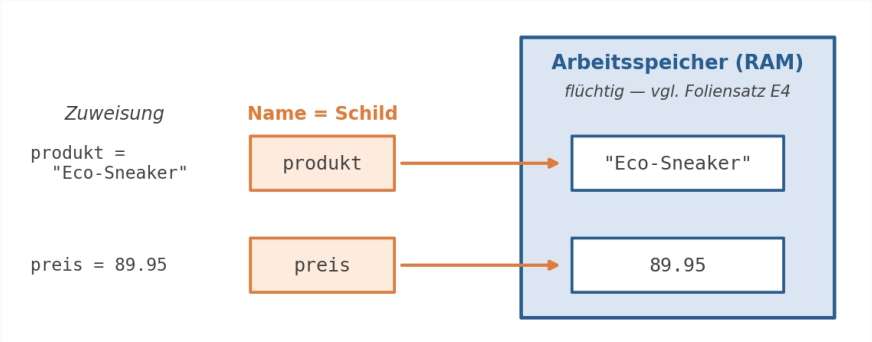
Zuweisung mit =

```
produktname = "Eco-Sneaker"
preis = 89.95
bestand = 42

print(produktname)      # Eco-Sneaker
print(preis)            # 89.95
```

- Links: der **Name**, rechts: der **Wert**
- Gelesen wird: *“preis **bekommt** 89.95”*
- Ab jetzt steht der Name überall dort, wo sonst der Wert stünde
- Python merkt selbst, ob Zahl oder Text gespeichert wird

Unter der Haube: Name → Wert im Speicher



Eine Zuweisung tut **zwei Dinge**: sie legt den Wert im **Arbeitsspeicher** ab (dem RAM aus Foliensatz E4) und heftet ein **Namensschild** daran. Ihr Programm hat jederzeit einen **Zustand**: die Tabelle *Name → Wert*.

Unter der Haube: = ist keine Gleichung

```
preis = 89.95       # OK: Wert bekommt den Namen
89.95 = preis       # SyntaxError!
```

So arbeitet Python **jede** Zuweisung ab:

Schritt	Python tut
1	**rechte** Seite auswerten → ein Wert
2	Wert im Speicher ablegen
3	Schild (Name) von **links** daran heften

- In Mathe ist $x = 5$ symmetrisch – in Python fließt der Wert strikt von **rechts nach links**
- Links darf deshalb **nur ein Name** stehen: `89.95 = preis` scheitert, an eine Zahl kann man kein Schild heften

Zuweisung – Demo & Übung

▶ **Live-Demo** – `01_erste_variable.py`

✎ **Sofort ausprobieren** – Notebook 04, Kap. 1: `lieblingsfarbe`, `punktzahl`, `vorname`/`nachname`

Neu zuweisen: Werte ändern

```
bestand = 42                # Morgens
bestand = 37                # 5 Paar verkauft
bestand = bestand - 1       # noch 1 Paar verkauft
print(bestand)              # 36
```

- Ein Name kann **beliebig oft** neu zugewiesen werden
- Der alte Wert verliert sein Schild – er ist **weg**
- `bestand = bestand - 1`: rechts steht der **aktuelle** Wert

Der **Name bleibt**, der **Wert wechselt** – deshalb heißt es *Variable*.

Unter der Haube: Neuzuweisung als Trace

```
x = 5
x = x + 1
print(x)        # 6
```

Schritt	Python tut	x danach
1	5 ablegen, Schild x anheften	5
2	rechte Seite: x + 1 $\rightarrow$ 5 + 1 $\rightarrow$ 6	noch 5
3	Schild x an die neue 6 umhängen	6

Immer **rechts zuerst**: die rechte Seite wird komplett mit dem **alten** Wert ausgewertet, erst dann wandert das Schild. Die alte 5 hat kein Schild mehr – Python räumt sie weg.

Neuzuweisung – Demo & Übung

▶ **Live-Demo** – `02_werte_aendern.py`

✎ **Sofort ausprobieren** – Notebook 04, Kap. 2: `stadt` von Berlin → München, `leben` von 3 auf 2

Variablennamen – was ist erlaubt?

Erlaubt

- Buchstaben, Ziffern, Unterstrich _
- Beginn mit Buchstabe oder _
- Groß-/Kleinschreibung zählt: `Preis` $\neq$ `preis`

Verboten

- Beginn mit Ziffer: `1produkt` – **SyntaxError**
- Leerzeichen und Sonderzeichen: `mein produkt`, `preis$`
- Reservierte Wörter: `class`, `if`, `for`, `def`

> Python prüft nur die **Regeln** – ob der Name etwas *bedeutet*, prüfen Ihre Leser:innen.

Konventionen für gute Namen

Stil	Beispiel	Wofür
`snake_case`	`produkt_name`, `auf_lager`	normale Variablen
`GROSSBUCHSTABEN`	`MWST_SATZ`, `WAEHRUNG`	Konstanten
sprechend	`kunden_email` statt `ke`	immer!

```
x = 89.95                # schlecht: was ist x?
preis_eco = 89.95        # gut: sagt, was drinsteht
```

Aussagekräftige Namen sind die wichtigste Form von Dokumentation – besser als jeder Kommentar (Notebook 03).

Namen – Demo & Übung

▶ **Live-Demo** – `03_namen_konventionen.py`

✎ **Sofort ausprobieren** – Notebook 04, Kap. 3: erlaubte vs. verbotene Namen erst raten, dann testen

Mehrfachzuweisung: ein Wert, drei Schilder

```
verkauft_mo = verkauft_di = verkauft_mi = 0
verkauft_mo = 3        # nur Mo bekommt neuen Wert
```

- Praktisch, um mehrere Zähler auf **denselben Startwert** zu setzen
- Unter der Haube: **ein** Wert `0` im Speicher, **drei** Schilder daran
- Eine Neuzuweisung hängt nur **ein** Schild um – `verkauft_di` und `verkauft_mi` bleiben `0`

▶ **Live-Demo** – `04_mehrfachzuweisung.py`

✎ **Sofort ausprobieren** – Notebook 04, Kap. 4: drei Zähler `start`, `mitte`, `ende` auf `0` (Aufgabe 2)

f-Strings: Variablen im Text

```
produkt = "Eco-Sneaker"
preis = 89.95
print(f"{produkt} kostet {preis} EUR")
```

Ausgabe: `Eco-Sneaker kostet 89.95 EUR`

- Das `f` vor dem Anführungszeichen macht den Text zum **f-String**
- In `{...}` steht ein **Variablenname** – eingesetzt wird sein **Wert**
- Ohne `f` bleibt `{produkt}` wörtlich im Text stehen

Unter der Haube: der f-String beim Ausführen

```
zeile = f"{produkt}: {preis} EUR"
```

Baustein	Python macht daraus
`{produkt}`	Schild nachschlagen → `Eco-Sneaker`
`: und EUR`	bleibt wörtlich stehen
`{preis}`	Schild nachschlagen → `89.95`
Ergebnis	**neuer** String `"Eco-Sneaker: 89.95 EUR"`

Ausgewertet wird **beim Ausführen der Zeile**: der f-String liest die Werte von **jetzt** und baut daraus einen fertigen String. Ändert sich `preis` später, ändert sich `zeile` **nicht** mit.

f-Strings – Demo & Übung

▶ **Live-Demo** – `05_fstring_bon.py`

✎ **Sofort ausprobieren** – Notebook 04, Kap. 2, Aufgabe 3: begrüßung im f-String ausgeben

Klassiker: Variablen-Tausch & Mini-Bon – die Aufgabe

Die klassische Prüfungsaufgabe – nur mit Zuweisungen lösbar:

- Zwei Lager mit Stückzahlen: `lager_a = 12`, `lager_b = 30`
- Die Inhalte der beiden Variablen **tauschen**
- Danach: einen **Mini-Bon** ausgeben – Produkt und Preis als Variablen, formatiert per f-String

So soll es laufen

```
Vorher:  Lager A = 12 | Lager B = 30
Nachher: Lager A = 30 | Lager B = 12
--- Bon ---
Eco-Sneaker: 89.95 EUR
```

Klassiker: Variablen-Tausch – Live-Coding

▶ **Live-Coding ab leerem Editor** – Referenz: `90_klassiker_variablentausch.py`
Teilziele:

1. Lager-Variablen anlegen, “Vorher”-Zeile per f-String
2. Naiver Tausch `lager_a = lager_b` – im Kopf tracen: warum ist die 12 danach verloren?
3. **Dreieckstausch**: Hilfsvariable `hilf` rettet den Wert
4. Mini-Bon mit f-Strings ausgeben

✎ **Zum Selberlösen** – Handout *“Klassiker 04: Variablen-Tausch”* (`Klassiker/04_Klassiker_Variablentausch.pdf`). Vertiefung: Notebook 04, Trainingsaufgabe 6.

Cheat Card

Aufgabe	Syntax
Wert speichern	`name = wert`
Wert neu zuweisen	`name = neuer_wert`
Mit altem Wert weiterrechnen	`bestand = bestand - 1`
Ein Wert, mehrere Namen	`a = b = c = 0`
Konstante (Konvention)	`MWST_SATZ = 0.19`
Variable im Text	`f"{produkt}: {preis} EUR"`
Klassiker heute	Tausch: `hilf = a` → `a = b` → `b = hilf`

Faustregel: = heißt **“bekommt”**, nie “ist gleich”. Immer rechts zuerst auswerten, dann das Schild umhängen.

Ausblick: Notebook 05 – einfache Datentypen

Unser Bon kann anzeigen – aber noch nicht **rechnen**:

```
preis = 89.95                    # Zahl mit Komma?
bestand = 42                     # ganze Zahl?
anzahl = input("Wie viele? ")    # immer TEXT!
```

- Hinter jedem Wert steckt ein **Typ**: `int`, `float`, `str`, `bool`
- `input()` liefert **immer Text** – auch wenn `"3"` eingetippt wird
- Rechnen, runden, vergleichen: dafür brauchen wir **Typumwandlung** und **Operatoren**

Im nächsten Notebook: was diese Typen können – und wie man sie ineinander umwandelt.

Heute geübt

- ✓ Variable = **Namensschild** an einem Wert im Arbeitsspeicher
- ✓ Zuweisung =: rechts auswerten, ablegen, Schild anheften
- ✓ = ist **keine Gleichung** – `89.95 = preis` scheitert
- ✓ Neuzuweisung als **Trace**: `x = x + 1` Schritt für Schritt
- ✓ Namensregeln und `snake_case`-Konventionen
- ✓ f-Strings: `{name}` wird beim Ausführen durch den Wert ersetzt
- ✓ **Klassiker Variablen-Tausch** live gesehen

✎ **Ihre Aufgaben bis zur nächsten Woche:**

- Klassiker **Variablen-Tausch** eigenständig lösen (Handout)
- “Sofort ausprobieren”-Aufgaben im Notebook 04
- Trainingsmaterial: gestufte Aufgaben (Steckbrief mit `input()`, Adresse, Spitzname)